C# Programming for Indian Students — Chapter 1: Welcome to C#

Oct 31, 2026 · @Vikash

1.1 What you will learn

This chapter introduces C# (pronounced "C sharp") without assuming that you have ever written a program. By the end you will be able to:

- explain what C# is, who made it and why it is popular;
- name real places where C# is used, in India and worldwide;
- describe in simple steps how a C# program goes from your typing to the screen;
- write, read and explain your first program, and fix the most common beginner errors.

Installing Visual Studio and creating projects is covered in Chapter 2, so here you only need to read the programs and understand them. If you already have Visual Studio or the .NET SDK, you can type each program and run it as you go.

How each chapter is organised. Every chapter has the same parts: concept, programs with real output, a dry run, common errors, a mini project, practice questions with answers, MCQs and viva questions, and a summary. Every program in this book was compiled and run, and the output shown is the real output (tested with the .NET 8 SDK; the programs behave the same on .NET 10).

Vikash Kumar

1.2 What is C#?

C# (pronounced "C sharp") is a modern, object-oriented programming language created by Microsoft. A team led by Anders Hejlsberg designed it, and it was released in 2002 together with the first version of .NET. The "sharp" comes from the music symbol ♯, which raises a note: C# was meant to be a step up from C and C++. Today the language is on version 14, which ships with .NET 10.

A **programming language** is a way to give instructions to a computer. **.NET** is the platform (the ready-made tools and libraries) that runs C# programs on Windows, Linux and macOS.

Key features

- **Simple and readable.** The code looks close to English, so it is a good first serious language.

- **Object-oriented.** Programs are built from classes and objects, which you will study from Chapter 10.
- **Type-safe.** The compiler checks that you use numbers as numbers and text as text, so many mistakes are caught before the program runs.
- **Automatic memory management.** A **garbage collector** frees memory you no longer use, so you do not release it by hand as in C or C++.
- **Cross-platform.** One language for web, desktop, mobile, games and cloud.
- **Always improving.** New features arrive every year, and Microsoft supports each long-term release for years.

C# compared with Java and C++

Point	C#	Java	C++
Created by	Microsoft	Sun Microsystems (now Oracle)	Bjarne Stroustrup
File extension	`.cs`	`.java`	`.cpp`
Runs on	.NET runtime (CLR)	Java Virtual Machine (JVM)	Directly on the machine
Memory handling	Automatic (garbage collector)	Automatic (garbage collector)	Manual
Program starts at	`Main` method	`main` method	`main` function

Remember: if you already know a little Java, you will find C# very familiar. If you know nothing yet, that is fine, because this book starts from zero.

1.3 Where is C# used?

C# is a general-purpose language, so you will meet it in many kinds of software. The examples below are everyday situations, so you can picture what you will be able to build.

Area	Everyday example	C# tool or framework
Websites and web APIs	An online ticket-booking site or a bank's mobile-app server	ASP.NET Core
Games	Mobile and PC games with scripts for characters and scores	Unity game engine

Vikash Kumar

Area	Everyday example	C# tool or framework
Desktop software	A billing program for a shop, a school-fee manager	WPF, Windows Forms
Mobile apps	One app for both Android and iPhone	.NET MAUI
Cloud services	A service that sends an SMS when a UPI payment succeeds	Azure Functions
Business software	Software for a company's accounts, stock and payroll	.NET libraries

Banks, insurance companies, government portals and IT service companies commonly use .NET for business software, so C# skills are useful for placements and internships as well as for your own projects.

Remember: you do not need to know these frameworks now. In this book you first learn the language itself, because every framework above is written in C#.

1.4 How a C# program runs

A computer's processor understands only machine code (0s and 1s), not C#. So your program is translated in stages.

Analogy. Imagine you write a letter in Hindi and your friends live in many countries. You first translate it into one common language (English). Then in each country a local interpreter reads that English version aloud in the local language. C# works the same way: your code is first translated into a common language called IL, and then each computer translates IL into its own machine code.

The five steps

1. **You write source code** in a file ending with `.cs`, for example `Program.cs`.
2. **The compiler** (named Roslyn) checks your code for mistakes. If there are none, it translates the code into **IL** (Intermediate Language).
3. **The IL is saved** in a file called an **assembly**, which ends with `.dll` (or `.exe` on Windows).
4. **The CLR** (Common Language Runtime) loads the assembly when you run it.
5. **The JIT compiler** (Just-In-Time) inside the CLR converts the IL to machine code for your computer, and the processor runs it.

Vikash Kumar

```
Program.cs  -->  Compiler  -->  Program.dll (IL)  -->  CLR + JIT  -->
Machine code  -->  Output
(you write)      (checks)       (saved on disk)         (on run)
(screen)
```

Term	Meaning in one line
Source code	The C# text you type
Compiler	Checks the code and translates it to IL
IL	A common, half-way language that is not tied to one computer
CLR	The engine that runs your program and manages memory
JIT	Converts IL to machine code at run time
Compile-time error	A mistake found by the compiler before the program starts
Run-time error	A mistake that appears while the program is running

Remember: the compiler catches grammar mistakes (compile-time errors). It cannot catch a wrong idea, such as adding when you should subtract. That is why you also test your programs.

C# and .NET architecture at a glance

The five steps above are the short story. Now let us open the box. A C# program never talks to your hardware directly. It sits on top of a few layers, and each layer has one job.

Vikash Kumar

```
+-------------------------------------------------+
|  YOUR C# PROGRAM        (Program.cs)            |
+-------------------------------------------------+
|  BASE CLASS LIBRARY (BCL)                       |
|  Console, String, List, File, DateTime ...      |
+-------------------------------------------------+
|  CLR - Common Language Runtime                  |
|  JIT compiler | Garbage Collector | Type safety |
+-------------------------------------------------+
|  OPERATING SYSTEM       (Windows / Linux / macOS)|
+-------------------------------------------------+
|  HARDWARE               (CPU, RAM, disk)        |
+-------------------------------------------------+
```

Read the diagram from the top: your code asks the BCL for ready-made tools (like `Console.WriteLine`), and the CLR runs everything and manages memory. The operating system then talks to the hardware.

The key parts of .NET

Part	Full form	What it does (in one line)
Compiler (Roslyn)	.NET Compiler Platform	Translates your C# source code into IL
IL	Intermediate Language	A CPU-independent, half-way language that the compiler produces
Assembly	.dll or .exe file	The package that holds IL code plus metadata (a description of your types)
CLR	Common Language Runtime	The engine that loads and runs your assembly
JIT	Just-In-Time compiler	Part of the CLR; converts IL into machine code while the program runs
GC	Garbage Collector	Part of the CLR; frees memory of objects you no longer use
BCL	Base Class Library	Thousands of ready-made classes such as Console, String and File
CTS	Common Type System	The rules for all types, so C# int is the same as System.Int32

Vikash Kumar

Managed code. Code that runs under the CLR (all normal C# code) is called managed code, because the CLR manages its memory and safety. Code that runs outside the CLR, such as old C++ programs, is unmanaged code.

Inside the compiler: from source code to IL

A computer's processor understands only machine code (0s and 1s). It cannot read `Console.WriteLine`. So a translator is needed. For C#, this translator is the compiler, named **Roslyn**, and it ships inside the .NET SDK. It does not jump straight to machine code. It works in stages, much like you read a sentence: first words, then grammar, then meaning.

Take the single line `int age = 18;` and watch it travel through the compiler.

Stage	Name	What happens	For `int age = 18;`
1	Lexical analysis	Source text is cut into small pieces called tokens	`int` `age` `=` `18` `;`
2	Syntax analysis (parsing)	Tokens are checked against C# grammar and arranged in a tree	A variable declaration: type int, name age, value 18
3	Semantic analysis	Meaning is checked: names exist, types match	Is 18 allowed in an int? Yes. Is age declared twice? No
4	Code generation	Valid code is turned into IL and written to the assembly	IL instructions saved in the .dll file

This is why different mistakes give different error codes. A missing semicolon is caught in stage 2 (syntax error, for example CS1002). Writing `int x = "hello";` passes stage 2 but fails in stage 3 (semantic error, CS0029). If any stage fails, **no assembly is produced** and nothing runs.

What IL looks like. You never write IL, but it helps to see it once. The one-line program `Console.WriteLine("Hello, India!");` becomes roughly this (simplified for learning):

```
ldstr "Hello, India!"
call void [System.Console]System.Console::WriteLine(string)
ret
```

The instructions say: load the string, call WriteLine, return. Notice that IL does not mention Intel or ARM. That is the point: the same IL can run on any machine that has the .NET runtime.

Vikash Kumar

Inside the runtime: from IL to the screen

When you run `dotnet run` (or press F5), these things happen in order:

1. The operating system starts the .NET host, which finds and starts the CLR.
2. The CLR loads your assembly (the .dll) and reads its metadata.
3. It finds the entry point (the `Main` method, or the hidden one created by top-level statements).
4. The JIT compiler converts the IL of that method into machine code, just before it is first needed. Methods that are never called are never compiled.
5. The CPU runs the machine code. The CLR keeps watch: it allocates memory for objects, and the garbage collector later cleans up unused ones.
6. The output appears on the console and the program ends.

```
Program.cs --Roslyn--> Program.dll (IL + metadata) --CLR + JIT--> machine
code --> CPU --> output
   (you)      compile time                                  run time
```

Compile time vs run time. Everything left of the CLR happens at *compile time* (before the program runs, on the developer's machine). Everything after happens at *run time* (while the program is running). Syntax errors are compile-time errors. A crash such as dividing by zero is a run-time error. You will meet this again in Chapter 11.

Why two steps? Think of IL as a recipe written in English. Any kitchen (Windows, Linux, macOS) with a cook (the CLR) who knows English can make the dish, and the cook adapts it to the local stove (the CPU). Compare with C or C++, which compile straight to machine code for one platform only. And compare with Java, which has the same idea (source to bytecode to JVM). The JIT also gets smarter as the program runs: .NET first compiles quickly, then re-compiles frequently used (hot) methods with more optimisation. This is called tiered compilation.

Good to know (not for exams). Native AOT and ReadyToRun are options that compile IL to machine code *before* running, to make start-up faster. We will not use them in this book.

See it yourself: a runtime information program

Demo 1.A - Peek into the runtime

```
using System.Runtime.InteropServices;

Console.WriteLine(RuntimeInformation.FrameworkDescription);
Console.WriteLine(typeof(int).FullName);
Console.WriteLine(typeof(string).FullName);
Console.WriteLine(typeof(Console).Assembly.GetName().Name);
```

Vikash Kumar

Output (the first line shows the .NET version installed on the machine, so yours may differ)

```
.NET 8.0.33
System.Int32
System.String
System.Console
```

How it works. Line 1 asks the CLR which .NET it is running on. Lines 2 and 3 prove the CTS idea: the C# keyword `int` is just a nickname for `System.Int32`, and `string` for `System.String`. Line 4 shows that `Console` lives in the assembly called System.Console, part of the BCL.

Try it: open the `bin/Debug` folder after `dotnet build` and find your `.dll` file. That file is the IL assembly the compiler produced. Delete it, run `dotnet run` again, and watch it get rebuilt.

Common mistakes.

Wrong belief	Truth
The .exe contains machine code	It contains IL; machine code is created by the JIT at run time
The compiler runs the program	The compiler only translates; the CLR runs the program
A program that builds cannot crash	Compile-time checks cannot catch everything; run-time errors still happen
C# runs only on Windows	The same IL runs on Windows, Linux and macOS with .NET

Check yourself (answers below).

1. What is the name of the C# compiler, and what does it produce?
2. At which stage of the compiler is a missing semicolon caught?
3. What is the job of the JIT compiler?
4. Why is IL called platform-independent?
5. Is a divide-by-zero error a compile-time or run-time error?

Answers. (1) Roslyn; it produces an assembly (.dll or .exe) containing IL and metadata. (2) Syntax analysis (parsing). (3) It converts IL into machine code while the program runs, method by method. (4) Because IL is not tied to any CPU; the CLR on each machine converts it to that machine's code. (5) Run-time error.

Vikash Kumar

1.5 Your first program

Program 1.1: Hello, India!

```
using System;

namespace HelloApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, India!");
        }
    }
}
```

Output

```
Hello, India!
```

Line-by-line explanation

Line	What it means
`using System;`	Lets you use ready-made tools from the `System` library, such as `Console`. Every statement ends with a semicolon `;`.
`namespace HelloApp`	A namespace is a named folder for your code, which stops names from clashing.
`class Program`	In C#, all code lives inside a class. Think of a class as a container for your program.
`static void Main(string[] args)`	`Main` is where the program starts. `static` means it runs without creating an object, `void` means it returns nothing, and `string[] args` holds any words typed after the program name.
`{ }`	Curly braces mark where a block of code begins and ends.
`Console.WriteLine("Hello, India!");`	Prints the text inside the double quotes on the screen and moves to the next line.

Vikash Kumar

The shorter modern form. Since C# 9, and in the default templates of current Visual Studio and .NET, you can write the same program with **top-level statements**. The compiler creates the class and `Main` for you.

Program 1.2: Hello, India! (modern form)

```
Console.WriteLine("Hello, India!");
```

Output

```
Hello, India!
```

This book uses the short form from now on, because it lets you focus on the idea. In exams and interviews you may still be asked for the full form, so know both.

Write, WriteLine and escape sequences

`Console.Write` prints and stays on the same line. `Console.WriteLine` prints and then moves to a new line. An **escape sequence** starts with a backslash `\` and prints a special character.

Escape	Meaning
`\n`	New line
`\t`	Tab (a gap up to the next tab stop)
`\"`	A double quote inside text
`\\`	A single backslash

Program 1.3: Write, WriteLine and escapes

```
Console.WriteLine("Namaste");
Console.Write("Welcome to ");
Console.Write("C#");
Console.WriteLine();
Console.WriteLine("Name:\tAarav");
Console.WriteLine("City:\tPune");
Console.WriteLine("He said \"Hello\"");
```

Output

Vikash Kumar

```
Namaste
Welcome to C#
Name:   Aarav
City:   Pune
He said "Hello"
```

The gap after `Name:` is a tab, which here lines up to the next tab stop. The exact width can differ slightly on your screen.

1.6 Dry run and common errors

A **dry run** means following a program line by line on paper, writing down what appears on the screen after each line. It is the best way to understand a program, and examiners often ask for it.

Dry run of Program 1.3

Step	Statement executed	What happens	Screen after this step
1	`Console.WriteLine("Namaste");`	Prints Namaste, moves to the next line	`Namaste`
2	`Console.Write("Welcome to ");`	Prints, stays on the same line	`Welcome to`
3	`Console.Write("C#");`	Continues on the same line	`Welcome to C#`
4	`Console.WriteLine();`	Prints nothing, moves to the next line	(cursor on a new line)
5	`Console.WriteLine("Name:\tAarav");`	Prints Name:, a tab, then Aarav	`Name: Aarav`
6	`Console.WriteLine("City:\tPune");`	Same pattern	`City: Pune`
7	`Console.WriteLine("He said \"Hello\"");`	Each `\"` prints a quote mark	`He said "Hello"`

Common beginner errors. The messages below are the real messages the compiler printed when these mistakes were made.

Vikash Kumar

Mistake	Wrong code	Compiler message	Fix
Missing semicolon	`Console.WriteLine("Hello, India!")`	`error CS1002: ; expected`	Add `;` at the end of the statement
Wrong capital letter in `Console`	`console.WriteLine("Hello, India!");`	`error CS0103: The name 'console' does not exist in the current context`	Write `Console` with a capital C
Wrong capital letter in `WriteLine`	`Console.Writeline("Hello, India!");`	`error CS0117: 'Console' does not contain a definition for 'Writeline'`	Write `WriteLine` with a capital L

Remember: C# is **case-sensitive**. `Console` and `console` are different names, and so are `WriteLine` and `Writeline`. Read the first error message carefully, because it usually points to the real problem, and fix errors from the top down.

1.7 Mini project: Student Welcome Card

Problem. Your college wants a small program that prints a welcome card for a new student. In later chapters you will read the details from the keyboard; here we print fixed details.

Program 1.4: Student Welcome Card

```
Console.WriteLine("==================================");
Console.WriteLine("      STUDENT WELCOME CARD");
Console.WriteLine("==================================");
Console.WriteLine("Name\t: Aarav Sharma");
Console.WriteLine("Course\t: BCA, Semester 2");
Console.WriteLine("City\t: Pune");
Console.WriteLine("Goal\t: Learn \"C#\" and build apps");
Console.WriteLine("==================================");
Console.WriteLine("Welcome to the world of C#!");
```

Output

Vikash Kumar

```
==================================
      STUDENT WELCOME CARD
==================================
Name    : Aarav Sharma
Course  : BCA, Semester 2
City    : Pune
Goal    : Learn "C#" and build apps
==================================
Welcome to the world of C#!
```

How it works. Each `WriteLine` prints one line of the card. The `\t` aligns the colons, and `\"` prints quote marks around C#. No variables are needed yet, which makes this a good first program to type, run and change.

Try it yourself

1. Change the details to your own name, course and city.
2. Add a line for your favourite subject.
3. Draw a border using `*` characters instead of `=`.

1.8 Practice questions

Try each question on your own first. The answers follow immediately after the list.

Section A: Short answer

1. Who developed C#, and in which year was it first released?
2. What is the difference between `Console.Write` and `Console.WriteLine`?
3. Name the five steps a C# program goes through from source code to output.
4. Why must C# code be written inside a class?
5. What is the role of the `Main` method?

Section B: Predict the output

6. What does this program print?

```
Console.Write("A");
Console.Write("B");
Console.WriteLine("C");
Console.WriteLine("D");
```

7. What does this program print?

Vikash Kumar

```
Console.WriteLine("Hi\tthere");
Console.WriteLine("Line1\nLine2");
Console.WriteLine("C:\\Users\\Aarav");
```

Section C: Find the error

8. Find the mistake: `Console.WriteLine("Hello, India!")`
9. Find the mistake: `console.WriteLine("Hello, India!");`

Section D: Write the program

10. Print this pattern using `WriteLine`: a right-angled triangle of stars with 3 rows.
11. Print a card with your name, roll number and college.

Answers

1. Microsoft developed C#, with a team led by Anders Hejlsberg. It was first released in 2002 with .NET.
2. `Write` prints and stays on the same line. `WriteLine` prints and then moves to the next line.
3. Write source code, compile to IL, save as an assembly, load in the CLR, and convert to machine code with the JIT, then run.
4. C# is an object-oriented language, so all code belongs to a type, and a class is the basic one. (Top-level statements hide this, because the compiler creates the class for you.)
5. `Main` is the entry point: the CLR starts running your program from it.
6. Output:

```
ABC
D
```

7. Output:

```
Hi      there
Line1
Line2
C:\Users\Aarav
```

8. The semicolon is missing at the end of the statement (`error CS1002: ; expected`).
9. `console` must be written with a capital C: `Console` (`error CS0103`).
10. Program and output:

Vikash Kumar

```
Console.WriteLine("*");
Console.WriteLine("**");
Console.WriteLine("***");
```

```
*
**
***
```

11. One possible answer:

```
Console.WriteLine("Name   : Priya Nair");
Console.WriteLine("Roll No: 21");
Console.WriteLine("College: Government Engineering College");
```

```
Name   : Priya Nair
Roll No: 21
College: Government Engineering College
```

1.9 MCQs and viva questions

Multiple-choice questions (answers are listed after the questions)

1. C# was developed by:
   - (a) Sun Microsystems
   - (b) Microsoft
   - (c) Google
   - (d) Oracle
2. Which file extension does a C# source file use?
   - (a) `.java`
   - (b) `.cpp`
   - (c) `.cs`
   - (d) `.py`
3. Which method is the entry point of a C# program?
   - (a) `Start`
   - (b) `Run`
   - (c) `Main`
   - (d) `Begin`
4. Which statement prints text and moves to the next line?
   - (a) `Console.Write`

Vikash Kumar

   - (b) `Console.WriteLine`
   - (c) `Console.Print`
   - (d) `Console.Show`
5. What does the compiler produce from C# source code?
   - (a) Machine code only
   - (b) IL (Intermediate Language)
   - (c) Java bytecode
   - (d) HTML
6. Which part of .NET converts IL to machine code at run time?
   - (a) Linker
   - (b) Editor
   - (c) JIT compiler
   - (d) Debugger
7. Every statement in C# normally ends with:
   - (a) a full stop
   - (b) a colon
   - (c) a semicolon
   - (d) a comma
8. The escape sequence for a new line is:
   - (a) `\t`
   - (b) `\n`
   - (c) `\\`
   - (d) `\"`
9. C# is:
   - (a) case-insensitive
   - (b) case-sensitive
   - (c) neither
   - (d) only case-sensitive for numbers
10. The garbage collector in .NET:
    - (a) prints errors
    - (b) frees memory that is no longer used
    - (c) compiles the code
    - (d) saves files

MCQ answers: 1-b, 2-c, 3-c, 4-b, 5-b, 6-c, 7-c, 8-b, 9-b, 10-b.

Vikash Kumar

Viva questions (short oral questions with model answers)

1. *What is .NET?* A platform of tools and libraries, with a runtime, that runs and supports C# programs.
2. *What is the CLR?* The Common Language Runtime. It loads your program, converts IL to machine code and manages memory.
3. *What is IL?* Intermediate Language, the half-way code the C# compiler produces. It is not tied to one computer.
4. *What is the difference between a compile-time error and a run-time error?* A compile-time error is found by the compiler before the program starts. A run-time error happens while the program is running.
5. *Why do we write* `using System;` *?* To use the ready-made types of the `System` namespace, such as `Console`, without writing `System.Console` each time.
6. *Can C# run on Linux and macOS?* Yes. Modern .NET is cross-platform.
7. *What are top-level statements?* A short form, available since C# 9, in which the compiler creates the class and `Main` method for you.

1.10 Summary and what comes next

- C# is a modern, object-oriented language from Microsoft, first released in 2002 and still updated every year.
- A C# program is compiled to IL, loaded by the CLR, converted to machine code by the JIT, and then runs.
- `Console.WriteLine` prints a line, `Console.Write` stays on the same line, and escape sequences such as `\n`, `\t` and `\"` print special characters.
- C# is case-sensitive, and statements end with a semicolon.
- A dry run, tracing a program line by line, is the best way to understand it.

Proposed chapter map of this book (for your approval; it follows the shape of the Codekitab Java book, adapted to C#)

Chapter	Title	Main ideas
1	Welcome to C#	What C# is, how programs run, first program
2	Setting Up C#	Installing Visual Studio and the .NET SDK, creating and running projects
3	C# Fundamentals	Variables, data types, constants, comments
4	Operators and User Input	Arithmetic, relational and logical operators, `Console.ReadLine`, type conversion

Vikash Kumar

Chapter	Title	Main ideas
5	Decision Making	`if`, `else`, `switch`, conditions
6	Loops	`for`, `while`, `do-while`, `foreach`, patterns
7	Methods	Parameters, return values, overloading
8	Arrays	One- and two-dimensional arrays
9	Strings	String methods, formatting, validation
10	Object-Oriented Programming	Classes, objects, inheritance, polymorphism, interfaces
11	Exception Handling	`try`, `catch`, `finally`, custom exceptions
12	Collections	`List`, `Dictionary`, `HashSet`, simple LINQ
13	File Handling	Reading and writing text files
14	Real-World C# Projects	GST calculator, attendance report, simple banking and UPI-style payment programs

Next chapter: Chapter 2 shows how to install Visual Studio and the .NET SDK, create your first project, and run Program 1.1 yourself.

Vikash Kumar